Practical 5: MongoDB Integration and Schema Design with Mongoose

Objective	To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.
Architecture / Diagram	
Diagram	Express App | v Mongoose ODM | v MongoDB Database └── tasks collection { title, description, completed, createdAt } Schema Validation: required fields + default values enforced before any document reaches the database.
Problem Definition	
Problem Statement	• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose. • Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now). • Replace the in-memory array from Practical 4 with real Mongoose model operations. • Test all CRUD operations against the live database using Postman. • Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

Steps

1. Install Mongoose and create a .env file for the connection string:
   npm install mongoose dotenv

```
PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Important document\Desktop\All-AWDF- Practical\task-manager-p5> npm f
und
task-manager-p5
├── https://dotenvx.com
│   └── dotenv@18.0.3
└── https://opencollective.com/mongoose
    └── mongoose@9.10.2
```

2. Connect to MongoDB inside server.js:

```
const mongoose = require('mongoose');
require('dotenv').config();
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch((err) => console.error(err));
```

```
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/taskmanager';

mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected');
    app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
    process.exit(1);
  });
```

3. Define the Task schema and model in a separate models/Task.js file:

```
const taskSchema = new mongoose.Schema({
 title: { type: String, required: true },
 description: { type: String },
 completed: { type: Boolean, default: false },
 createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('Task', taskSchema);
```

```
const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Title is required'],
    trim: true
  },
  description: {
    type: String,
    default: ''
  },
  completed: {
    type: Boolean,
    default: false
  },
  priority: {
    type: String,
    enum: {
      values: ['low', 'medium', 'high'],
      message: '{VALUE} is not a valid priority. Must be low, medium, or high.'
    },
    default: 'medium'
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

taskSchema.pre('save', function (next) {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
  next();
});

module.exports = mongoose.model('Task', taskSchema);
```

4. Replace the in-memory array logic from Practical 4 with Mongoose methods: Task.find(), Task.create(), Task.findByIdAndUpdate(), Task.findByIdAndDelete().

task-manager-p5 > backend > controllers > taskController.js > ...

```
const Task = require('../models/Task');

exports.getAllTasks = async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: tasks.length, data: tasks });
  } catch (err) {
    next(err);
  }
};

exports.getTaskById = async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with id ${req.params.id}`
      });
    }
    res.status(200).json({ success: true, data: task });
  } catch (err) {
    next(err);
  }
};

exports.createTask = async (req, res, next) => {
  try {
    const task = await Task.create(req.body);
    res.status(201).json({ success: true, data: task });
  } catch (err) {
    next(err);
  }
};
```

```
exports.updateTask = async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true
    });
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with id ${req.params.id}`
      });
    }
    res.status(200).json({ success: true, data: task });
  } catch (err) {
    next(err);
  }
};


exports.deleteTask = async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with id ${req.params.id}`
      });
    }
    res.status(200).json({ success: true, data: {} });
  } catch (err) {
    next(err);
  }
};
```

5. Wrap each route in a try/catch and pass errors to the global error handler using next(err).

task-manager-p5 > backend > middleware > errorHandler.js > ...

```
function errorHandler(err, req, res, next) {
  console.error(err.stack);

  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map((val) => val.message);
    return res.status(400).json({
      success: false,
      error: messages
    });
  }

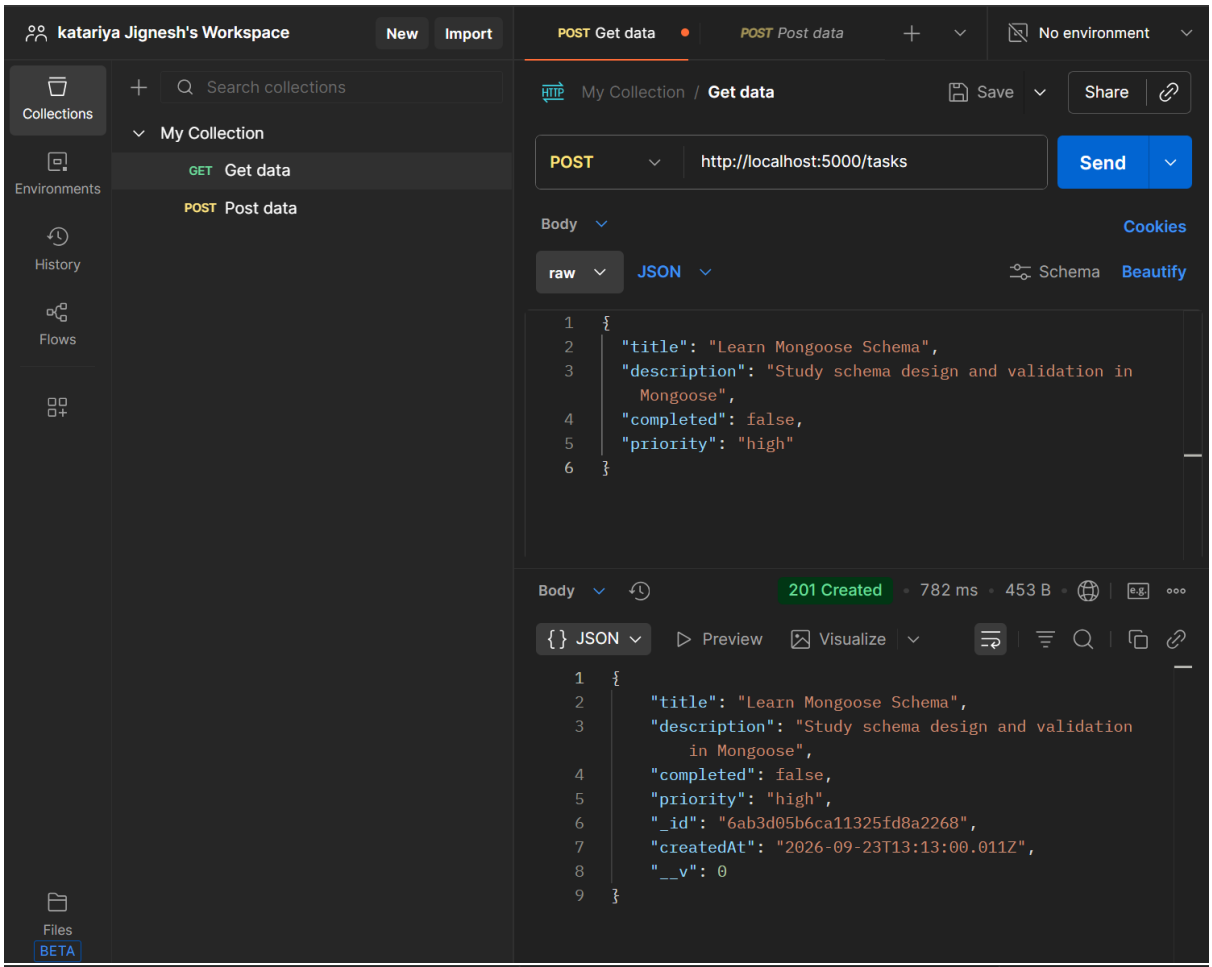
  if (err.name === 'CastError') {
    return res.status(400).json({
      success: false,
      error: `Invalid ${err.path}: ${err.value}`
    });
  }

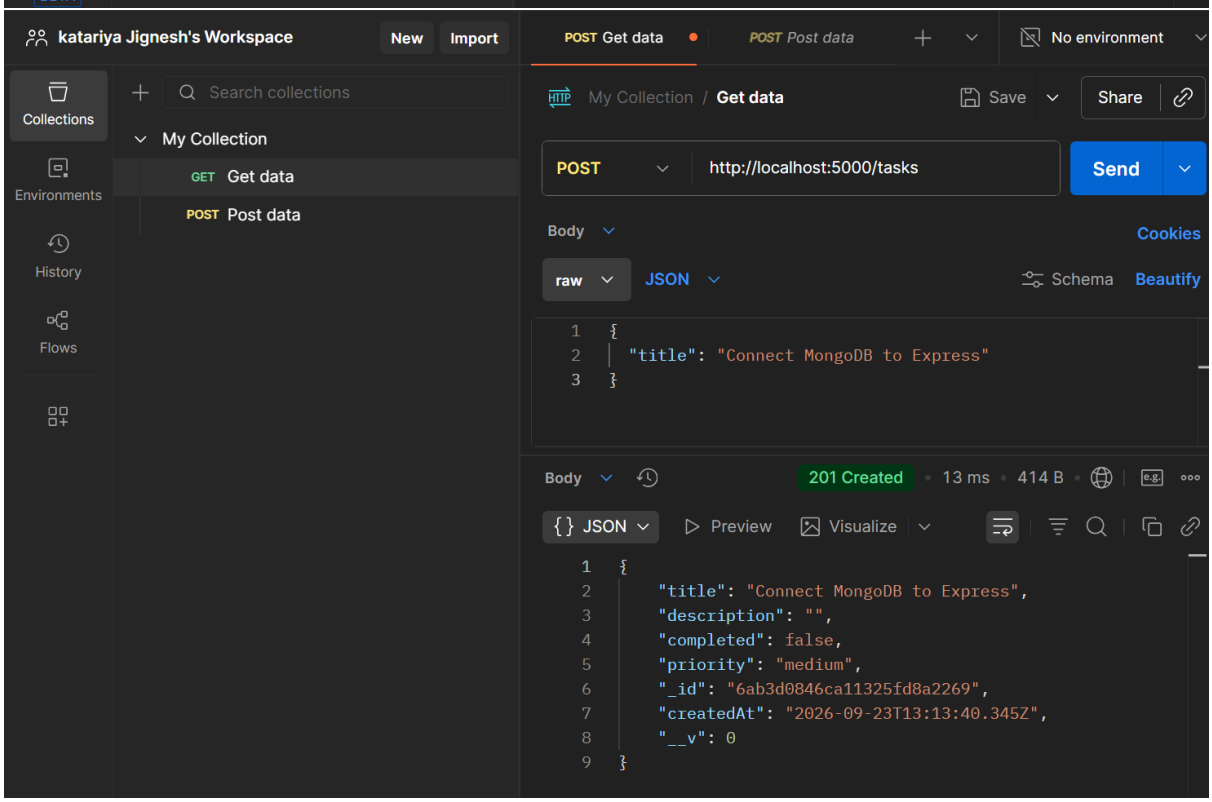
  if (err.code === 11000) {
    return res.status(400).json({
      success: false,
      error: 'Duplicate field value entered'
    });
  }

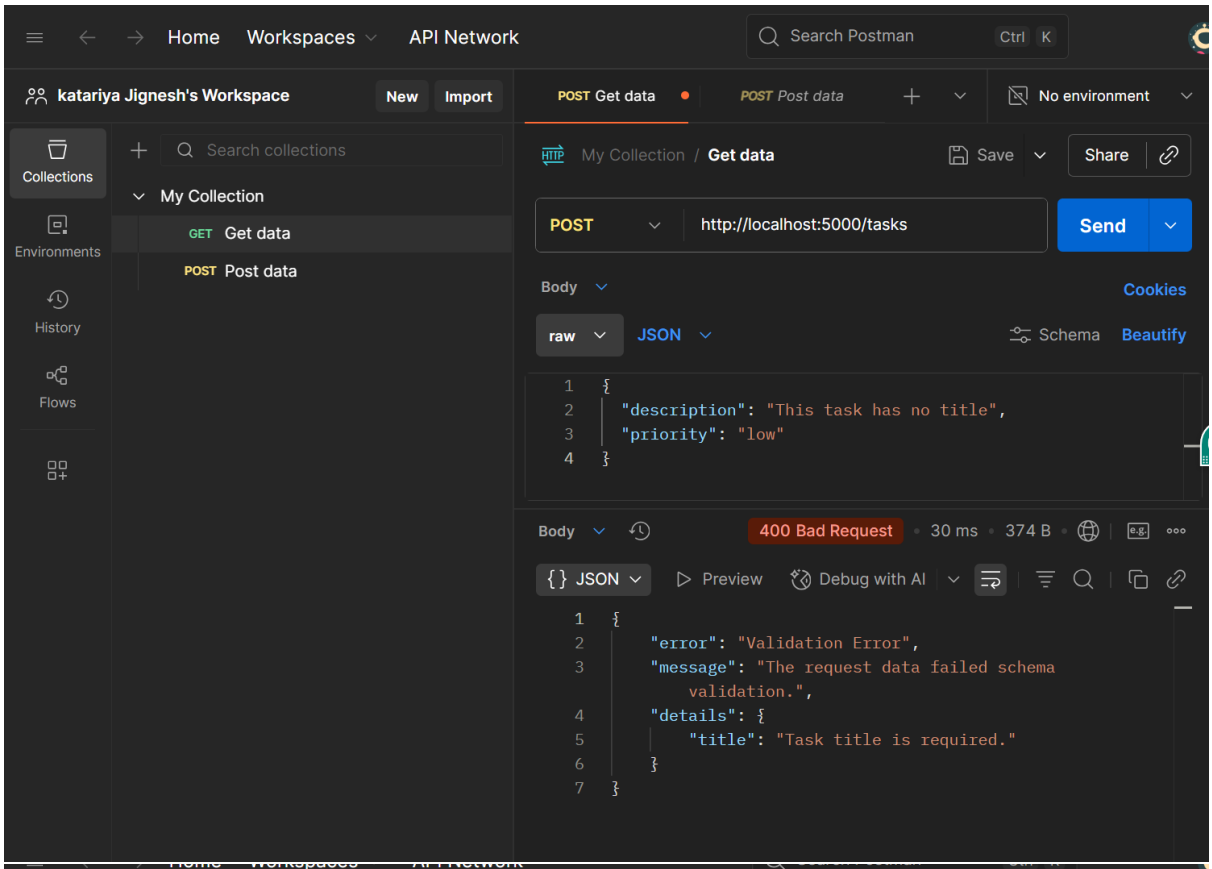
  res.status(err.statusCode || 500).json({
    success: false,
    error: err.message || 'Something went wrong'
  });
}

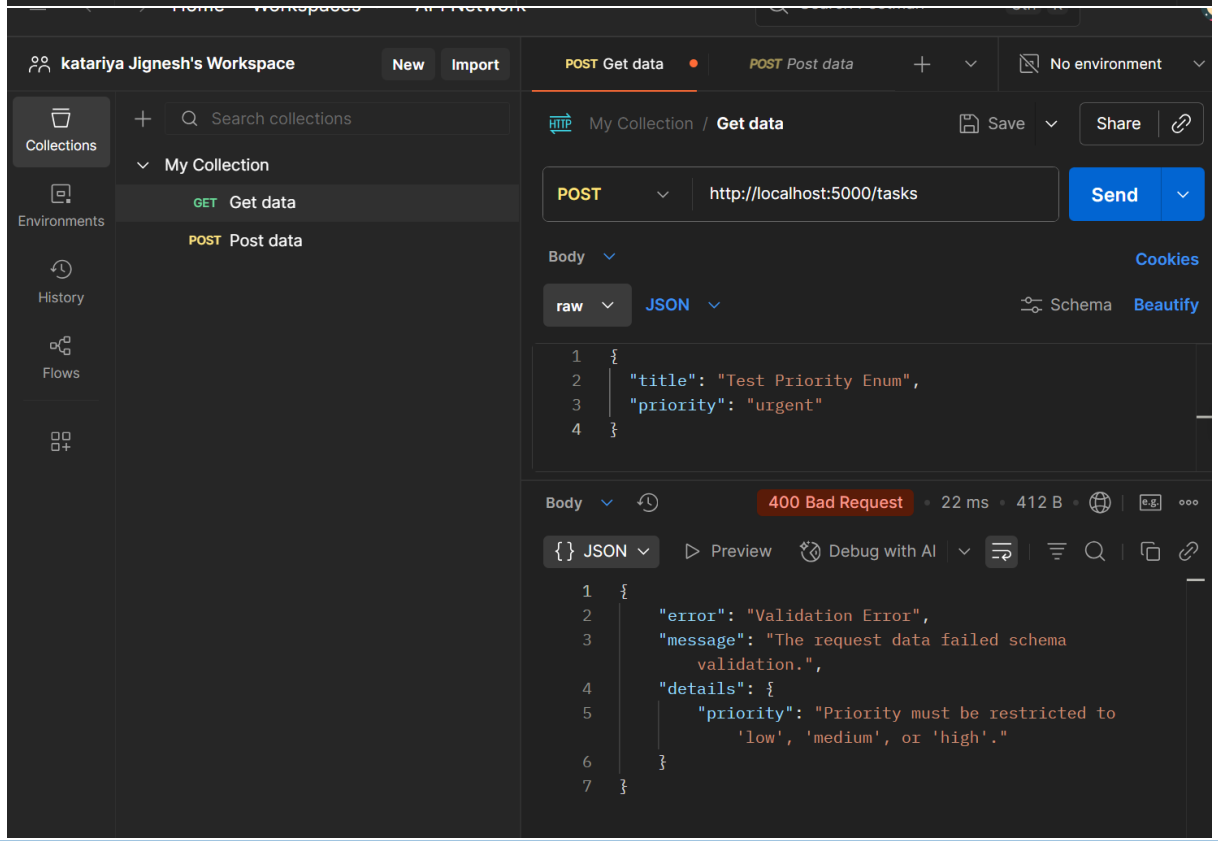
module.exports = errorHandler;
```

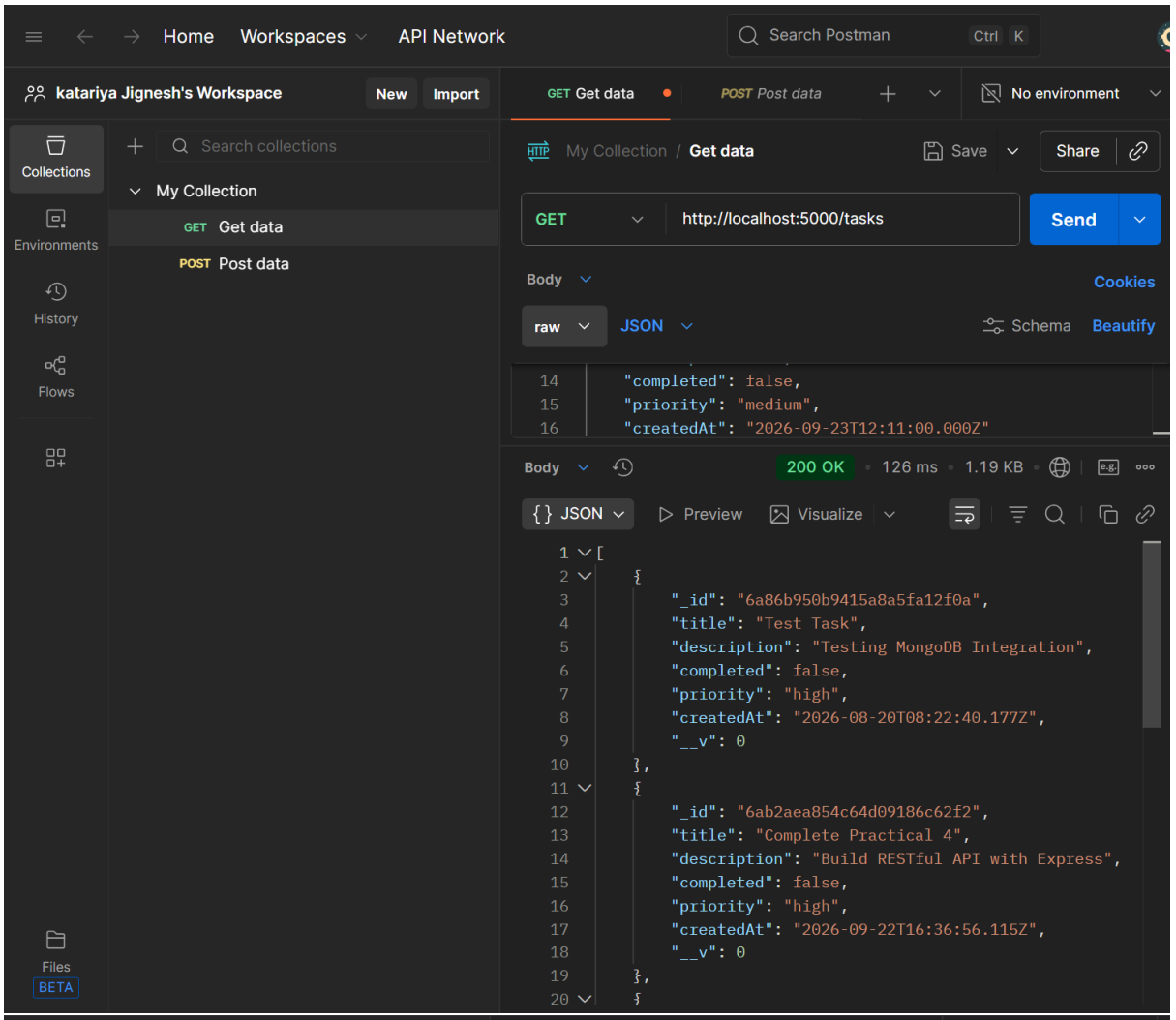
6. Test all 4 endpoints in Postman; confirm data persists in MongoDB after server restart

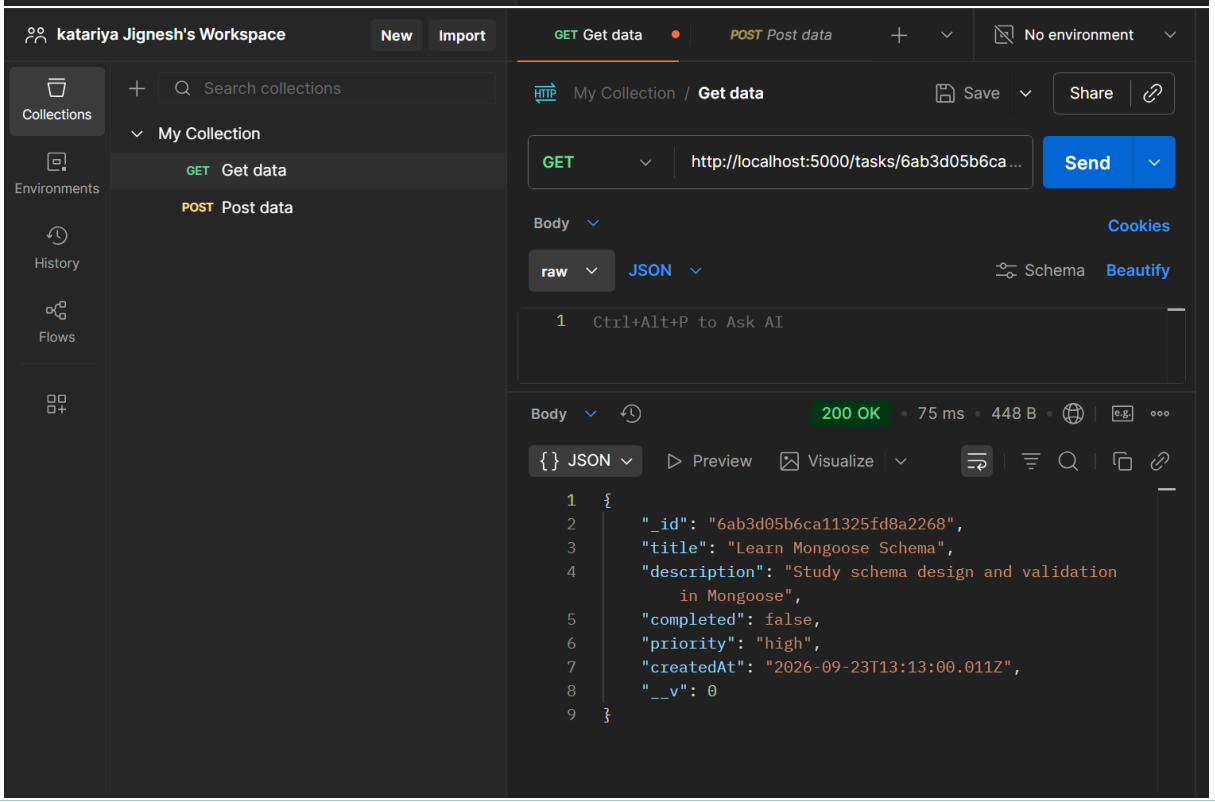

katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Files
BETA
Search collections
My Collection
GET Get data
POST Post data
POST Get data
POST Post data
No environment
My Collection / Get data
Save
Share
POST
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"title": "Learn Mongoose Schema",
"description": "Study schema design and validation in Mongoose",
"completed": false,
"priority": "high"
}
Body
201 Created
782 ms
453 B
{} JSON
Preview
Visualize
{
"title": "Learn Mongoose Schema",
"description": "Study schema design and validation in Mongoose",
"completed": false,
"priority": "high",
"_id": "6ab3d05b6ca11325fd8a2268",
"createdAt": "2026-09-23T13:13:00.011Z",
"__v": 0
}

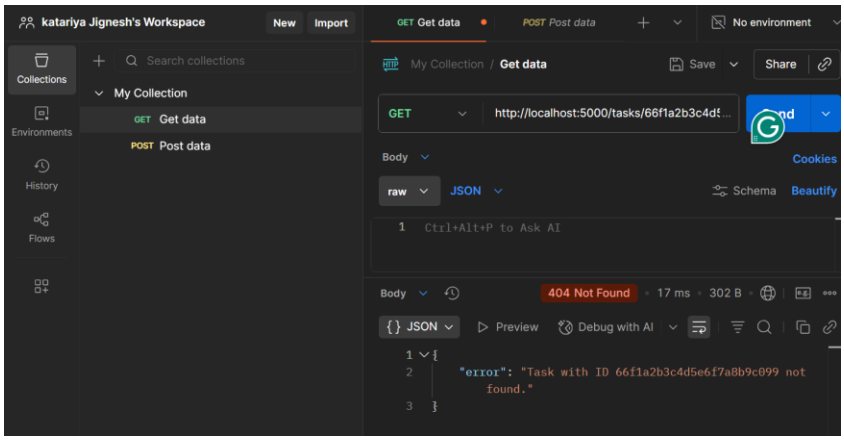

katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
POST Get data
POST Post data
No environment
My Collection / Get data
Save
Share
POST
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"title": "Connect MongoDB to Express"
}
Body
201 Created
13 ms
414 B
{} JSON
Preview
Visualize
{
"title": "Connect MongoDB to Express",
"description": "",
"completed": false,
"priority": "medium",
"_id": "6ab3d0846ca11325fd8a2269",
"createdAt": "2026-09-23T13:13:40.345Z",
"__v": 0
}

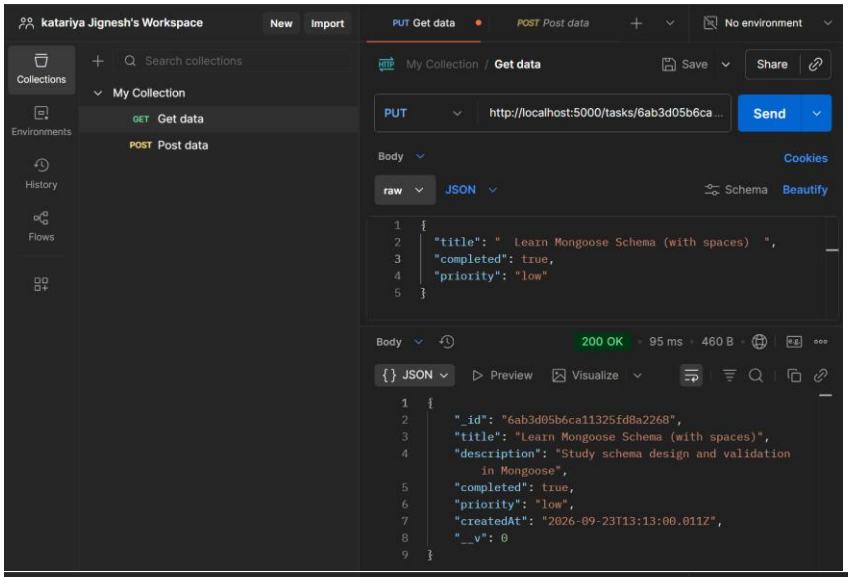

Home
Workspaces
API Network
Search Postman
Ctrl K
katariya Jignesh's Workspace
New
Import
POST Get data
POST Post data
No environment
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
My Collection / Get data
Save
Share
POST
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"description": "This task has no title",
"priority": "low"
}
Body
400 Bad Request
30 ms
374 B
JSON
Preview
Debug with AI
{
"error": "Validation Error",
"message": "The request data failed schema validation.",
"details": {
"title": "Task title is required."
}
}

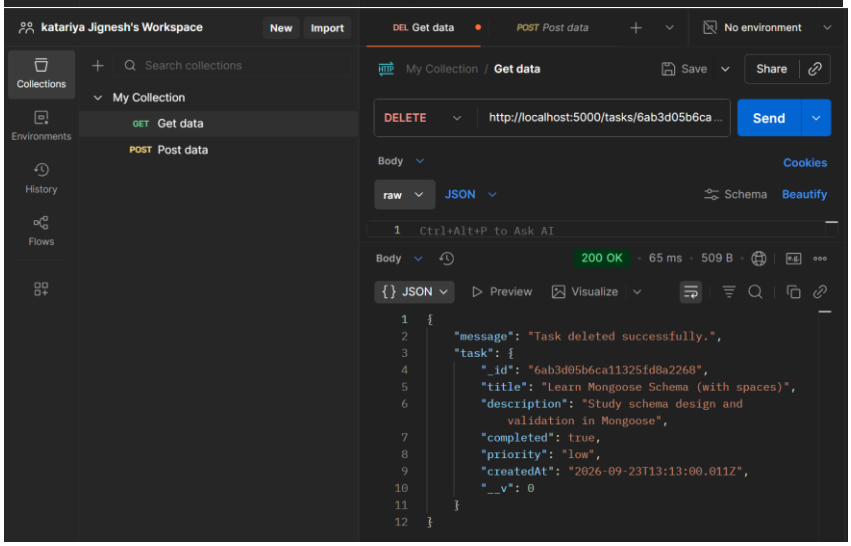

katariya Jignesh's Workspace
New
Import
POST Get data
POST Post data
No environment
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
My Collection / Get data
Save
Share
POST
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"title": "Test Priority Enum",
"priority": "urgent"
}
Body
400 Bad Request
22 ms
412 B
JSON
Preview
Debug with AI
{
"error": "Validation Error",
"message": "The request data failed schema validation.",
"details": {
"priority": "Priority must be restricted to 'low', 'medium', or 'high'."
}
}


Home
Workspaces
API Network
Search Postman
Ctrl K
katariya Jignesh's Workspace
New
Import
GET Get data
POST Post data
No environment
Collections
Environments
History
Flows
Files
BETA
Search collections
My Collection
GET Get data
POST Post data
My Collection / Get data
Save
Share
GET
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
"completed": false,
"priority": "medium",
"createdAt": "2026-09-23T12:11:00.000Z"
Body
200 OK
126 ms
1.19 KB
{} JSON
Preview
Visualize
[
{
"_id": "6a86b950b9415a8a5fa12f0a",
"title": "Test Task",
"description": "Testing MongoDB Integration",
"completed": false,
"priority": "high",
"createdAt": "2026-08-20T08:22:40.177Z",
"__v": 0
},
{
"_id": "6ab2aea854c64d09186c62f2",
"title": "Complete Practical 4",
"description": "Build RESTful API with Express",
"completed": false,
"priority": "high",
"createdAt": "2026-09-22T16:36:56.115Z",
"__v": 0
},


katariya Jignesh's Workspace
New
Import
GET Get data
POST Post data
No environment
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
My Collection / Get data
Save
Share
GET
http://localhost:5000/tasks/6ab3d05b6ca ...
Send
Body
Cookies
raw
JSON
Schema
Beautify
Ctrl+Alt+P to Ask AI
Body
200 OK
75 ms
448 B
{} JSON
Preview
Visualize
{
"_id": "6ab3d05b6ca11325fd8a2268",
"title": "Learn Mongoose Schema",
"description": "Study schema design and validation in Mongoose",
"completed": false,
"priority": "high",
"createdAt": "2026-09-23T13:13:00.011Z",
"__v": 0
}


katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
GET Get data
POST Post data
No environment
My Collection / Get data
Save
Share
GET
http://localhost:5000/tasks/66f1a2b3c4d5
Body
Cookies
raw
JSON
Schema
Beautify
1 Ctrl+Alt+P to Ask AI
Body
404 Not Found
17 ms
302 B
{} JSON
Preview
Debug with AI
{
"error": "Task with ID 66f1a2b3c4d5e6f7a8b9c099 not found."
}


katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
PUT Get data
POST Post data
No environment
My Collection / Get data
Save
Share
PUT
http://localhost:5000/tasks/6ab3d05b6ca
Send
Body
Cookies
raw
JSON
Schema
Beautify
{
"title": " Learn Mongoose Schema (with spaces) ",
"completed": true,
"priority": "low"
}
Body
200 OK
95 ms
460 B
{} JSON
Preview
Visualize
{
"_id": "6ab3d05b6ca11325fd8a2268",
"title": "Learn Mongoose Schema (with spaces)",
"description": "Study schema design and validation in Mongoose",
"completed": true,
"priority": "low",
"createdAt": "2026-09-23T13:13:00.011Z",
"__v": 0
}


katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
DEL Get data
POST Post data
No environment
My Collection / Get data
Save
Share
DELETE
http://localhost:5000/tasks/6ab3d05b6ca
Send
Body
Cookies
raw
JSON
Schema
Beautify
1 Ctrl+Alt+P to Ask AI
Body
200 OK
65 ms
509 B
{} JSON
Preview
Visualize
{
"message": "Task deleted successfully.",
"task": {
"_id": "6ab3d05b6ca11325fd8a2268",
"title": "Learn Mongoose Schema (with spaces)",
"description": "Study schema design and validation in Mongoose",
"completed": true,
"priority": "low",
"createdAt": "2026-09-23T13:13:00.011Z",
"__v": 0
}
}

Key Questions / Analysis	**1. Purpose of a schema in MongoDB if it is schema-less?** **MongoDB is schema-less, but Mongoose Schema defines the expected structure, data types, and rules for documents. It provides consistency and validation.** **2. Why define required fields and default values at schema level?** **Because frontend validation can be bypassed or fail. Schema-level validation works on the server, ensuring invalid or missing data cannot easily be stored in the database.** **3. What happens when a document fails Mongoose validation? Where is the request stopped?** **Mongoose checks the document before saving it. If validation fails, Mongoose generates a ValidationError, and the document is not saved to MongoDB. The error is passed back to the application, where error-handling middleware can send the response.** **Flow:** **Client Request** **↓** **Express Route** **↓** **Mongoose Validation ✗** **↓** **ValidationError** **↓** **Error Middleware** **↓** **Response to Client**